
Backend – Lab Session 1

Edition: 1

Issuing Date: 03 February 2025

Objective

Create a small API using .NET 8 to learn and use the following topics:

- Building REST APIs
- API Documentation
- Input Validation
- Backend Configuration
- Dependency Injection

Building REST APIs and API Documentation

Steps

1. Open your IDE create a new ASP .NET core Web API.
2. Create a class called User that have 3 properties:
 1. Long Id
 2. String name
 3. Email
3. Create a list of users and fill it manually with random data.
4. Create an endpoint that returns all the users.
5. Create a GET endpoint that takes a route param an Id of a user and it returns the user with the Id from the request.
6. Create a GET endpoint that have from query param a string value (name filter) and returns all the users whose name contain the value.
7. Create a GET endpoint that returns the current Date formatted base on the Accepted -Language Header
 1. en-US
 2. es-ES
 3. fr-FR

Hint: CultureInfo using System.Globalization

8. Create a POST endpoint to change a user's name based on the Id the endpoint has a body containing 3 properties:
 1. The Id of the user we wish to update
 2. The new name.
 3. Email
9. Create a POST endpoint that use FromForm to upload an image and save it in a directory (it would be better if inside the wwwroot directory)

Hint: Use IFormFile

10. Create a DELETE endpoint that deletes a user from the list.

Input Validation and Exception Handling

Add to the endpoints you created above a validation ex: Delete Endpoints requires an Id to function.

Throw exceptions base on each type of validation.

In the endpoint created in step 7 add to it a try catches finally block that checks if the Accepted Language sent is a valid one. Hint: .NET has function that can returns all supported Cultures.

Dependency Injection

Create a service that contains the functions that should be executed in the endpoints Injected in the controller and call the respective functions in each endpoint.

Register the Service in container choose between Singleton, transient, Scoped.